



Diseño Orientado a Objetos 2 (OO2)

Resumen y material de la materia



Calendario y Contenido

Semana	Fechas	Temas Principales	Material
1	9 mar - 15 mar	Introducción a Refactoring	Ver
2	16 mar - 22 mar	Refactoring de código	Ver
3	23 mar - 29 mar	<i>Feriado</i>	-
4	30 mar - 5 abr	Introducción a Patrones de Diseño. Adapter y Template Method	Ver
5	6 abr - 12 abr	Patrones: Composite, Factory Method y Builder	Ver
6	13 abr - 19 abr	Patrones: State y Strategy	Ver
7	20 abr - 26 abr	TDD (Test Driven Development)	Ver
8	28 abr - 3 may	Patrones: Decorator y Proxy	Ver
9	4 may - 10 may	Refactoring to Patterns	Ver
10	11 may - 17 may	Frameworks de Caja Blanca	Ver
11	18 may - 24 may	Frameworks de Caja Negra. FrozenSpot, Hotspots y Patrones	Ver



Índice de Temas

1. Refactorización (Refactoring)

- Introducción a Refactoring
- Técnicas de Refactoring de código
- Refactoring hacia patrones (*Refactoring to Patterns*)

2. Patrones de Diseño

- Introducción a los Patrones de Diseño
- **Patrones Creacionales:**
 - Factory Method
 - Builder
- **Patrones Estructurales:**
 - Adapter
 - Composite

- Decorator
- Proxy
- **Patrones de Comportamiento:**
 - Template Method
 - State
 - Strategy

3. Metodologías de Desarrollo

- TDD (Test Driven Development - Desarrollo guiado por pruebas)

4. Frameworks

- Conceptos generales y diferencias con bibliotecas
- Frameworks de Caja Blanca
- Frameworks de Caja Negra
- Consideraciones de Diseño:
 - Frozenspots
 - Hotspots
- Relación entre Frameworks y Patrones de Diseño



1. Refactorización (Refactoring)

La refactorización es una transformación del código que preserva su comportamiento, pero mejora su diseño. Sus objetivos principales son ganar compresión del código, reducir el costo de mantenimientos, facilitar detección de bugs y permitir agregar nuevas funcionalidades más rápidamente. Se basa en la premisa de no sobre-diseñar, sino mantener el código CLEAN (cohesivo, bajo acoplamiento, encapsulado, no redundante). Una regla de oro es la metáfora de los 2 sombreros: al programar se alterna entre el sombrero de "Agregar Funcionalidad" y el de "Refactorizar", recordando que solo se puede refactorizar cuando los tests están en verde (pasando). Es fundamental también comprender el concepto de Deuda Técnica (Technical Debt): tomar prestado tiempo en el diseño permite ganar feedback rápido, pero el "interés" de esa deuda es el esfuerzo extra que requiere modificar el sistema arrastrando un mal diseño. Por eso, es importante refactorizar continuamente para "pagar" esa deuda y mantener el código saludable a largo plazo.

Deuda Técnica (Technical Debt)

Es una metáfora introducida por Ward Cunningham para explicar a los stakeholders la necesidad de refactorizar. Es aceptable "tomar prestado" tiempo de diseño para entregar código rápido, ganar feedback y aprender en el corto plazo. El peligro radica en no pagar esa deuda.

- Capital de la deuda: El costo estimado de remediar los problemas de diseño (es decir, el esfuerzo que requerirá el refactoring masivo). Herramientas como SonarQube ayudan a cuantificarlo.
- Interés de la deuda: El costo adicional o esfuerzo extra que se paga en cada nueva tarea debido a tener que lidiar con un mal diseño. Si la deuda no se paga, los intereses se acumulan y ralentizan el desarrollo.

Bad Smells

Los Bad Smells (malos olores) son indicios en el código que sugieren problemas de diseño y señalan la necesidad de aplicar refactorizaciones. El catálogo de Fowler los clasifica en cinco grandes grupos:

1. Bloaters (Hinchas): Elementos o métodos que han crecido tanto que se han vuelto inmanejables.

- Método Largo (Long Method): Métodos con demasiadas líneas de código (complejos y difíciles de reusar):
- Refactorización: Extract Method (Extraer Método), Decompose Conditional (Descomponer Condicional), Replace Temp with Query (Reemplazar Variable Temporal por Consulta).
- Clase Grande (Large Class): Clases con demasiados métodos o variables de instancia, lo que indica baja cohesión.
- Refactorización: Extract Class (Extraer Clase), Extract Subclass (Extraer Subclase).
- Lista Larga de Parámetros (Long Parameter List): Métodos que requieren una larga lista de parámetros, dificultando su comprensión y uso.
- Refactorización: Introduce Parameter Object (Introducir Objeto Parámetro), Preserve Whole Object (Preservar Objeto Completo).

2. Dispensables (Prescindibles): Elementos innecesarios cuyo código debería ser limpiado o eliminado.

- Código Duplicado (Duplicate Code): Código idéntico o muy similar en múltiples lugares, lo que dificulta el mantenimiento.
- Refactorización: Extract Method (Extraer Método), Pull Up Method (Subir Método), Form Template Method (Formar Método Plantilla).
- Clase de Datos (Data Class): Clases que solo contienen datos sin comportamiento, lo que puede llevar a la "envidia de atributo".
- Refactorización: Move Method (Mover Método), Move Field (Mover Campo).
- Generalidad Especulativa (Speculative Generality): Introducir ganchos, clases o parámetros abstractos asumiendo cambios futuros que nunca ocurren.
- Refactorización: Collapse Hierarchy (Colapsar Jerarquía), Remove Parameter (Eliminar Parámetro).

3. Couplers (Acopladores): Infracciones que generan un acoplamiento excesivo entre clases.

- Envidia de Atributo (Feature Envy): Un método de una clase pasa más tiempo enviando mensajes y usando datos de otra clase que de la suya propia. Indica que el método está en el lugar equivocado.
- Refactorización: Move Method (Mover Método), Move Field (Mover Campo).
- Cadena de Mensajes (Message Chains): Llamadas encadenadas a través de múltiples objetos, lo que hace que el código sea difícil de entender y mantener.
- Refactorización: Hide Delegate (Ocultar Delegado)

4. Tool Abusers (Abusadores de Orientación a Objetos): Elementos que abusan de las características de la orientación a objetos, como la herencia o el polimorfismo.

- Condicionales o Sentencias Switch (Switch Statements): Uso excesivo de condicionales para diferenciar tipos de objetos o comportamientos, lo que sugiere la necesidad de aplicar polimorfismo.
- Refactorización: Replace Conditional with Polymorphism (Reemplazar Condicional por Polimorfismo), Replace Conditional Logic with Strategy (Reemplazar Lógica Condicional por Strategy).

- Legado Rechazado (Refused Bequest): Cuando las subclases heredan métodos o propiedades que no necesitan, lo que indica una jerarquía de clases mal diseñada.
- Refactorización: Extract Subclass (Extraer Subclase), Extract Superclass (Extraer Superclase).
- Temporary Variable (Variable Temporal): Variables que se utilizan para almacenar resultados intermedios o estados temporales, lo que puede indicar la necesidad de extraer métodos o clases.
- Refactorización: Extract Method (Extraer Método), Replace Temp with Query (Reemplazar Variable Temporal por Consulta), Inline Temporary Variable (Incorporar Variable Temporal).

5. Change Preventers (Impedidores del Cambio): Malos olores que rompen el principio de cambio en un solo lugar, encareciendo las modificaciones.

- Cambio Divergente (Divergent Change): Una sola clase se ve modificada de distintas formas ante diferentes tipos de requerimientos (baja cohesión).
 - Refactorización: Extract Class (Extraer Clase), Extract Subclass (Extraer Subclase).
- Cirugía de Escopeta (Shotgun Surgery): Al realizar un único cambio en el sistema, te ves obligado a hacer pequeñas modificaciones en muchas clases distintas.
- Refactorización: Move Method (Mover Método), Move Field (Mover Campo).

Metodos de Refactorización

Los métodos de refactorización son técnicas específicas que se aplican para mejorar el diseño del código sin alterar su comportamiento. Algunos de los métodos más comunes incluyen:

1. Composición de Metodos: Ayudan a distribuir adecuadamente el código y a lidiar con métodos muy largos o completos:

- Extract Method (Extraer Método):
 - Cuando usarlo: Cuando un método es muy largo o realiza varias tareas, por lo general para los bad smells de Método Largo, Código Duplicado y Variable Temporal.
 - Como aplicarlo: Se crea un nuevo método con un nombre descriptivo y se reemplaza el código original por una llamada a este nuevo método. Esto mejora la legibilidad y facilita la reutilización del código.
- Replace Temp with Query (Reemplazar Variable Temporal por Consulta):
 - Cuando usarlo: Cuando una variable temporal se utiliza para almacenar un valor que puede ser calculado mediante una consulta o método.
 - Como aplicarlo: Se reemplaza la variable temporal con una llamada a un método que realiza la misma operación, eliminando la necesidad de almacenar el valor en una variable temporal.

2. Mover Aspectos entre objetos: Ayudan a mejorar la asignación de responsabilidades (cohesión) y a resolver problemas de acoplamiento:

- Move Method (Mover Método):
 - Cuando usarlo: Cuando un método parece estar más relacionado con otra clase que con la clase en la que se encuentra actualmente (por ejemplo, en casos de Envidia de Atributo).
 - Como aplicarlo: Se mueve el método a la clase donde tiene más sentido, actualizando las referencias y asegurando que el comportamiento del código se mantenga igual.

- Move Field (Mover Campo):
- Cuando usarlo: Cuando un campo (variable de instancia) parece estar más relacionado con otra clase que con la clase en la que se encuentra actualmente.
- Como aplicarlo: Se mueve el campo a la clase donde tiene más sentido, actualizando las referencias y asegurando que el comportamiento del código se mantenga igual.

3. Manipulacion de la generalizacion: Sirven para mejorar el diseño de jerarquias de clases

- Pull up Method (Subir Método):
- Cuando usarlo: Cuando un método es idéntico en varias subclases y puede ser generalizado en la superclase.
- Como aplicarlo: Se mueve el método a la superclase, eliminando las implementaciones duplicadas en las subclases.
- Pull up Field (Subir Campo):
- Cuando usarlo: Cuando un campo es idéntico en varias subclases y puede ser generalizado en la superclase.
- Como aplicarlo: Se mueve el campo a la superclase, eliminando las implementaciones duplicadas en las subclases.
- Extract Superclass (Extraer Superclase):
- Cuando usarlo: Cuando varias clases comparten atributos y métodos comunes, lo que sugiere la necesidad de una superclase.
- Como aplicarlo: Se crea una nueva superclase que contiene los atributos y métodos comunes, y se hace que las clases existentes hereden de esta superclase.
- Extract Subclass (Extraer Subclase):
- Cuando usarlo: Cuando una clase tiene atributos y métodos que solo son relevantes para un subconjunto de sus instancias, lo que sugiere la necesidad de una subclase.
- Como aplicarlo: Se crea una nueva subclase que contiene los atributos y métodos específicos, y se hace que las instancias relevantes de la clase original sean instancias de la subclase.

4. Simplificacion de Expresiones Condicionales:

- Replace Conditional with Polymorphism (Reemplazar Condicional por Polimorfismo):
- Cuando usarlo: Cuando se tienen múltiples condicionales que dependen del tipo de objeto, lo que sugiere que se podría utilizar polimorfismo para simplificar el código.
- Como aplicarlo: Se crea una jerarquía de clases donde cada subclase implementa su propio comportamiento, eliminando la necesidad de condicionales.

5. Refactorizacion hacia patrones: Introducir patrones de diseño recién cuando el software evoluciona al punto en que son necesarios, en lugar de intentar adivinarlos o diseñarlos de antemano de forma especulativa (evitando así el over-engineering o sobre-diseño) Ejemplos de esto son Form Template Method, Replace Conditional Logic with Strategy, o Move Embellishment to Decorator.

- Form Template Method (Formar Método Plantilla):
- Cuando usarlo: Cuando se tiene un método que contiene una serie de pasos que pueden variar en las subclases, lo que sugiere la necesidad de un patrón Template Method.
- Como aplicarlo: Se crea un método plantilla en la superclase que define la estructura general del algoritmo, y se permite que las subclases implementen los pasos específicos.
- Replace conditional Logic with Strategy (Reemplazar Lógica Condicional por Strategy):

- Cuando usarlo: Cuando se tienen múltiples condicionales que determinan el comportamiento de un objeto, lo que sugiere la necesidad de un patrón Strategy.
- Como aplicarlo: Se crea una interfaz Strategy y varias implementaciones concretas. El objeto utiliza una instancia de Strategy para delegar el comportamiento, eliminando la necesidad de condicionales.
- Replace State-Altering Conditionals with State (Reemplazar Condicionales que Alteran Estado por State):
- Cuando usarlo: Cuando se tienen condicionales que cambian el comportamiento de un objeto según su estado, lo que sugiere la necesidad de un patrón State.
- Como aplicarlo: Se crea una interfaz State y varias implementaciones concretas. El objeto mantiene una referencia a su estado actual y delega el comportamiento a la instancia de State correspondiente, eliminando la necesidad de condicionales.
- Introduce Null Object (Introducir Objeto Nulo):
- Cuando usarlo: Cuando se tiene un objeto que puede ser nulo y se realizan múltiples verificaciones de nulidad, lo que sugiere la necesidad de un patrón Null Object.
- Como aplicarlo: Se crea una clase Null Object que implementa la misma interfaz que el objeto real, pero proporciona un comportamiento predeterminado. El código cliente puede utilizar el Null Object en lugar de realizar verificaciones de nulidad.

⚡ 2. Patrones de Diseño

Refactorizar hacia patrones consiste en introducir patrones de diseño recién cuando el software evoluciona al punto en que son necesarios, en lugar de intentar adivinarlos o diseñarlos de antemano de forma especulativa (evitando así el over-engineering o sobre-diseño) Ejemplos de esto son Form Template Method, Replace Conditional Logic with Strategy, o Move Embellishment to Decorator.

Patrones Vistos

🔧 Patrones Creacionales

Se enfocan en abstraer y flexibilizar los procesos de instanciación de los objetos.

- **Factory Method:** Define una interfaz para crear objetos, pero le da a sus subclasses la decisión de qué clase específica instanciar. Permite a las subclasses decidir que clase concreta se debe instanciar, además permite agregar nuevas clases concretas sin modificar el código existente (Las nuevas clases deben implementar la interfaz del producto). Ejemplo:

```
// Interfaz del producto
public interface Transporte {
    void entregar();
}

// Productos concretos
public class Camion implements Transporte {
    @Override
    public void entregar() {
        System.out.println("Entrega realizada por camión");
    }
}
```

```
public class Barco implements Transporte {
    @Override
    public void entregar() {
        System.out.println("Entrega realizada por barco");
    }
}

// Clase creadora
public abstract class Logistica {
    public abstract Transporte crearTransporte();

    public void planificarEntrega() {
        Transporte transporte = crearTransporte();
        transporte.entregar();
    }
}

// Creadores concreto
public class LogisticaTerrestre extends Logistica {
    @Override
    public Transporte crearTransporte() {
        return new Camion();
    }
}

public class LogisticaMaritima extends Logistica {
    @Override
    public Transporte crearTransporte() {
        return new Barco();
    }
}

// Uso del patrón
public class Main {
    public static void main(String[] args) {
        System.out.println("--- Prueba de Logística Terrestre ---");
        Logistica logisticaTerrestre = new LogisticaTerrestre();
        logisticaTerrestre.planificarEntrega();
        // Salida: Entregando carga por tierra en una caja de camión.

        System.out.println("\n--- Prueba de Logística Marítima ---");
        Logistica logisticaMaritima = new LogisticaMaritima();
        logisticaMaritima.planificarEntrega();
        // Salida: Entregando carga por mar en un contenedor marítimo.
    }
}

// Si se quisiera agregar un transporte entonces...
public class Avion implements Transporte {
    @Override
    public void entregar() {
        System.out.println("Entrega realizada por avión");
    }
}
```

```

    }

    public class LogisticaAerea extends Logistica {
        @Override
        public Transporte crearTransporte() {
            return new Avion();
        }
    }

    // Y listo para usarse
    public class Main {
        public static void main(String[] args) {
            System.out.println("--- Prueba de Logística Terrestre ---");
            Logistica logisticaTerrestre = new LogisticaTerrestre();
            logisticaTerrestre.planificarEntrega();
            // Salida: Entregando carga por tierra en una caja de camión.

            System.out.println("\n--- Prueba de Logística Marítima ---");
            Logistica logisticaMaritima = new LogisticaMaritima();
            logisticaMaritima.planificarEntrega();
            // Salida: Entregando carga por mar en un contenedor marítimo.

            System.out.println("\n--- Prueba de Logística Aérea ---");
            Logistica logisticaAerea = new LogisticaAerea();
            logisticaAerea.planificarEntrega();
            // Salida: Entregando carga por aire en un avión.
        }
    }
}

```

- **Builder:** El patrón Builder nos permite crear diferentes 'sabores' (configuraciones) de un objeto complejo siguiendo siempre la misma 'receta' (proceso de construcción), paso a paso. Separa la construcción de un objeto complejo de su representación final, de manera que el mismo proceso de construcción pueda crear implementaciones o configuración diferentes. Ejemplo:

```

// Clase Producto
public class Casa {
    private String paredes;
    private String techo;
    private int ventanas;
    private boolean tieneGaraje;
    private boolean tienePiscina;

    public void setParedes(String paredes) { this.paredes = paredes; }
    public void setTecho(String techo) { this.techo = techo; }
    public void setVentanas(int ventanas) { this.ventanas = ventanas; }
    public void setTieneGaraje(boolean tieneGaraje) { this.tieneGaraje =
tieneGaraje; }
    public void setTienePiscina(boolean tienePiscina) { this.tienePiscina =
tienePiscina; }

    @Override

```



```
        public String toString() {
            return "Casa con paredes de " + paredes + ", techo de " + techo +
                ", " + ventanas + " ventanas" +
                (tieneGaraje ? ", con Garaje" : ", sin Garaje") +
                (tienePiscina ? ", con Piscina." : ", sin Piscina.");
        }
    }

    // Interfaz del Builder
    public interface CasaBuilder {
        void construirParedes();
        void construirTecho();
        void construirVentanas();
        void construirGaraje();
        void construirPiscina();
        Casa getCasa();
    }

    // Builder concreto
    public class CasaModernaBuilder implements CasaBuilder {
        private Casa casa;

        public CasaModernaBuilder() {
            this.casa = new Casa();
        }

        @Override
        public void construirParedes() { casa.setParedes("concreto y vidrio"); }
        @Override
        public void construirTecho() { casa.setTecho("plano"); }
        @Override
        public void construirVentanas() { casa.setVentanas(10); }
        @Override
        public void construirGaraje() { casa.setTieneGaraje(true); }
        @Override
        public void construirPiscina() { casa.setTienePiscina(true); }

        @Override
        public Casa getCasa() { return this.casa; }
    }

    public class CadaAntiguaBuilder implements CasaBuilder {
        private Casa casa;

        public CadaAntiguaBuilder() {
            this.casa = new Casa();
        }

        @Override
        public void construirParedes() { casa.setParedes("ladrillo"); }
        @Override
        public void construirTecho() { casa.setTecho("tejas"); }
        @Override
        public void construirVentanas() { casa.setVentanas(4); }
```

```
@Override
public void construirGaraje() { casa.setTieneGaraje(false); }
@Override
public void construirPiscina() { casa.setTienePiscina(false); }

@Override
public Casa getCasa() { return this.casa; }
}

// Director (Quien controla la construcción)
public class Director {
    private CasaBuilder builder;

    public Director(CasaBuilder builder) {
        this.builder = builder;
    }

    public void construirCasa() {
        builder.construirParedes();
        builder.construirTecho();
        builder.construirVentanas();
        builder.construirGaraje();
        builder.construirPiscina();
    }

    public Casa getCasa() { return builder.getCasa(); }
}

// Uso del patrón
public class Main {
    public static void main(String[] args) {
        // Construcción de una casa moderna
        CasaBuilder modernaBuilder = new CasaModernaBuilder();
        Director directorModerna = new Director(modernaBuilder);
        directorModerna.construirCasa();
        Casa casaModerna = directorModerna.getCasa();
        System.out.println("Casa Moderna: " + casaModerna);

        // Construcción de una casa antigua
        CasaBuilder antiguaBuilder = new CasaAntiguaBuilder();
        Director directorAntigua = new Director(antiguaBuilder);
        directorAntigua.construirCasa();
        Casa casaAntigua = directorAntigua.getCasa();
        System.out.println("Casa Antigua: " + casaAntigua);
    }
}
```

Patrones Estructurales

Se enfocan en cómo las clases y objetos se componen para formar estructuras más grandes.

- **Adapter:** Permite que interfaces incompatibles trabajen juntas. Actúa como un puente entre dos interfaces, convirtiendo la interfaz de una clase en otra que el cliente espera. Ejemplo:

```
// Interfaz existente (Target)
public interface Enchufe {
    void conectar();
}

// Clase existente (Adaptee)
public class EnchufeEuropeo {
    public void enchufarEuropeo() {
        System.out.println("Conectado a enchufe europeo.");
    }
}

// Adaptador
public class AdaptadorEnchufe implements Enchufe {
    private EnchufeEuropeo enchufeEuropeo;

    public AdaptadorEnchufe(EnchufeEuropeo enchufeEuropeo) {
        this.enchufeEuropeo = enchufeEuropeo;
    }

    @Override
    public void conectar() {
        enchufeEuropeo.enchufarEuropeo();
    }
}

// Uso del patrón
public class Main {
    public static void main(String[] args) {
        Enchufe enchufe = new AdaptadorEnchufe(new EnchufeEuropeo());
        enchufe.conectar();
        // Salida: Conectado a enchufe europeo.
    }
}
```

- **Composite:** Permite tratar objetos individuales y composiciones de objetos de manera uniforme. Se utiliza para representar jerarquías de objetos en forma de árbol, donde los nodos pueden ser tanto hojas (objetos individuales) como compuestos (objetos que contienen otros objetos). Ejemplo:

```
// Componente
public interface Componente {
    void mostrar();
}

// Hoja
public class Hoja implements Componente {
```

```
private String nombre;

public Hoja(String nombre) {
    this.nombre = nombre;
}

@Override
public void mostrar() {
    System.out.println("Hoja: " + nombre);
}
}

// Compuesto
public class Compuesto implements Componente {
    private String nombre;
    private List<Componente> componentes = new ArrayList<>();

    public Compuesto(String nombre) {
        this.nombre = nombre;
    }

    public void agregar(Componente componente) {
        componentes.add(componente);
    }

    @Override
    public void mostrar() {
        System.out.println("Compuesto: " + nombre);
        for (Componente componente : componentes) {
            componente.mostrar();
        }
    }
}

// Uso del patrón
public class Main {
    public static void main(String[] args) {
        Compuesto arbol = new Compuesto("Árbol");
        Hoja hoja1 = new Hoja("Hoja 1");
        Hoja hoja2 = new Hoja("Hoja 2");
        Compuesto rama = new Compuesto("Rama");
        Hoja hoja3 = new Hoja("Hoja 3");

        arbol.agregar(hoja1);
        arbol.agregar(hoja2);
        rama.agregar(hoja3);
        arbol.agregar(rama);

        arbol.mostrar();
    }
}
```

- **Decorator:** Permite agregar responsabilidades adicionales a un objeto de manera dinámica. Se utiliza para extender la funcionalidad de un objeto sin modificar su estructura. Ejemplo:

```
// 1. COMPONENTE (La Interfaz Base)
// Define el contrato estricto. Tanto el café original como los ingredientes
extra
// DEBEN implementar esto. Así garantizamos el Polimorfismo (el cliente los
trata igual).
public interface Cafe {
    String descripcion();
    double costo();
}

// 2. COMPONENTE CONCRETO (El objeto base)
// Es el núcleo de nuestro pedido. Es el objeto que va a ser "envuelto" o
"decorado".
public class CafeSimple implements Cafe {
    @Override
    public String descripcion() {
        return "Café simple";
    }

    @Override
    public double costo() {
        return 2.0;
    }
}

// 3. DECORADOR BASE (La clave del patrón)
// Es una clase abstracta que hace dos cosas fundamentales:
// A. Implementa la interfaz (para poder disfrazarse de un Café).
// B. Guarda una referencia al objeto que está envolviendo (Composición).
public abstract class CafeDecorador implements Cafe {
    // Referencia protegida al componente envuelto.
    protected Cafe cafeEnvuelto;

    public CafeDecorador(Cafe cafe) {
        this.cafeEnvuelto = cafe;
    }

    // Por defecto, delega el trabajo al objeto que tiene adentro.
    @Override
    public String descripcion() {
        return cafeEnvuelto.descripcion();
    }

    @Override
    public double costo() {
        return cafeEnvuelto.costo();
    }
}
```

```
// 4. DECORADORES CONCRETOS (Los agregados)
// Extienden el decorador base para inyectar su propio comportamiento ANTES o
// DESPUÉS
// de delegar la llamada al objeto original.

public class ConLeche extends CafeDecorador {
    public ConLeche(Cafe cafe) {
        super(cafe);
    }

    @Override
    public String descripcion() {
        // Ejecuta la lógica del objeto envuelto y LE SUMA su propia
        descripción
        return super.descripcion() + ", con leche";
    }

    @Override
    public double costo() {
        // Ejecuta la lógica del objeto envuelto y LE SUMA su propio costo
        return super.costo() + 0.5;
    }
}

public class ConChocolate extends CafeDecorador {
    public ConChocolate(Cafe cafe) {
        super(cafe);
    }

    @Override
    public String descripcion() {
        return super.descripcion() + ", con chispas de chocolate";
    }

    @Override
    public double costo() {
        return super.costo() + 0.8;
    }
}

// 5. USO DEL PATRÓN (El Cliente)
public class Main {
    public static void main(String[] args) {

        System.out.println("--- Pedido 1: Café Básico ---");
        Cafe miCafe = new CafeSimple();
        System.out.println(miCafe.descripcion() + " - Costo: $" +
        miCafe.costo());
        // Salida: Café simple - Costo: $2.0

        System.out.println("\n--- Pedido 2: Agregando Leche ---");
        // Envolvemos el café simple dentro del decorador ConLeche
        miCafe = new ConLeche(miCafe);
        System.out.println(miCafe.descripcion() + " - Costo: $" +
```

```

miCafe.costo());
    // Salida: Café simple, con leche - Costo: $2.5

    System.out.println("\n--- Pedido 3: Agregando Chocolate (Apilamiento)
    ---");
    // Envolvemos el café (que ya tiene leche) dentro del decorador
    ConChocolate.
    // ¡Esto evita la explosión de subclases! No necesitamos una clase
    "CafeConLecheYChocolate".
    miCafe = new ConChocolate(miCafe);
    System.out.println(miCafe.descripcion() + " - Costo: $" +
    miCafe.costo());
    // Salida: Café simple, con leche, con chispas de chocolate - Costo:
    $3.3
    }
}

```

- **Proxy:** Proporciona un sustituto o representante de otro objeto para controlar el acceso a él. Se utiliza para agregar una capa de control, como la gestión de recursos, la seguridad o la carga diferida. Ejemplo:

```

// Interfaz del sujeto
public interface Imagen {
    void mostrar();
}

// Sujeto Real
public class ImagenReal implements Imagen {
    private String nombreArchivo;

    public ImagenReal(String nombreArchivo) {
        this.nombreArchivo = nombreArchivo;
        cargarImagenDesdeDisco();
    }

    private void cargarImagenDesdeDisco() {
        System.out.println("Cargando imagen desde disco: " + nombreArchivo);
    }

    @Override
    public void mostrar() {
        System.out.println("Mostrando imagen: " + nombreArchivo);
    }
}

// Proxy
public class ProxyImagen implements Imagen {
    private ImagenReal imagenReal;
    private String nombreArchivo;

    public ProxyImagen(String nombreArchivo) {

```

```
        this.nombreArchivo = nombreArchivo;
    }

    @Override
    public void mostrar() {
        if (imagenReal == null) {
            imagenReal = new ImagenReal(nombreArchivo);
        }
        // Delegar la llamada al objeto real
        // Esto asegura que la imagen solo se cargue cuando sea necesario
        (carga diferida)
        // Y permite controlar el acceso a la imagen real.
        imagenReal.mostrar();
    }
}
```

Patrones de Comportamiento

- **Template Method:** Define el esqueleto de un algoritmo en una operación, delegando algunos pasos a subclases. Permite que las subclases redefinan ciertos pasos del algoritmo sin cambiar su estructura.

Ejemplo:

```
// Clase abstracta que define el esqueleto del algoritmo
public abstract class Juego {
    // Método plantilla que define la secuencia de pasos
    public final void jugar() {
        iniciar();
        while (!finDelJuego()) {
            jugarTurno();
        }
        finalizar();
    }

    protected abstract void iniciar();
    protected abstract void jugarTurno();
    protected abstract boolean finDelJuego();
    protected abstract void finalizar();
}

// Subclase concreta que implementa los pasos específicos del juego
public class Ajedrez extends Juego {
    private int turno = 0;
    private final int maxTurnos = 10;

    @Override
    protected void iniciar() {
        System.out.println("Iniciando partida de ajedrez...");
    }

    @Override
    protected void jugarTurno() {
```



```

        System.out.println("Turno " + (turno + 1) + ": Jugando ajedrez...");
        turno++;
    }

    @Override
    protected boolean finDelJuego() {
        return turno >= maxTurnos;
    }

    @Override
    protected void finalizar() {
        System.out.println("Partida de ajedrez finalizada.");
    }
}

// Uso del patrón
public class Main {
    public static void main(String[] args) {
        Juego juego = new Ajedrez();
        juego.jugar();
        // Salida:
        // Iniciando partida de ajedrez...
        // Turno 1: Jugando ajedrez...
        // ...
        // Turno 10: Jugando ajedrez...
        // Partida de ajedrez finalizada.
    }
}

```

- **Strategy:** Define una familia de algoritmos, los encapsula en objetos separados y los hace intercambiables. Esto permite que el algoritmo varíe independientemente (en tiempo de ejecución) del contexto o los clientes que lo usan, eliminando condicionales complejos. Ejemplo:

```

// Interfaz para la estrategia
public interface Estrategia {
    void ejecutar();
}

// Implementación concreta de la estrategia
public class EstrategiaConcretaA implements Estrategia {
    @Override
    public void ejecutar() {
        System.out.println("Ejecutando estrategia A");
    }
}

// Implementación concreta de la estrategia
public class EstrategiaConcretaB implements Estrategia {
    @Override
    public void ejecutar() {
        System.out.println("Ejecutando estrategia B");
    }
}

```

```

    }
}

// Contexto que utiliza la estrategia
public class Contexto {
    private Estrategia estrategia;

    public void setEstrategia(Estrategia estrategia) {
        this.estrategia = estrategia;
    }

    public void ejecutarEstrategia() {
        estrategia.ejecutar();
    }
}

// Uso del patrón
public class Main {
    public static void main(String[] args) {
        Contexto contexto = new Contexto();
        contexto.setEstrategia(new EstrategiaConcretaA());
        contexto.ejecutarEstrategia();
        // Salida: Ejecutando estrategia A

        contexto.setEstrategia(new EstrategiaConcretaB());
        contexto.ejecutarEstrategia();
        // Salida: Ejecutando estrategia B
    }
}

```

- **State:** Permite a un objeto alterar drásticamente su comportamiento dependiendo de cuál sea su estado interno. Hace que los cambios de estado sean explícitos y elimina las sentencias condicionales largas atadas al estado del objeto. A diferencia de strategy, es necesario conocer los estados posibles del objeto y el cambio de estado es controlado por el objeto mismo, no por el cliente. Esto permite que el objeto cambie su comportamiento de manera más autónoma y encapsule la lógica de transición de estados dentro de sus propias clases de estado. Ejemplo:

```

// Interfaz para el estado
public interface Estado {
    void manejar(Contexto contexto);
}

// Implementación concreta del estado A
public class EstadoConcretoA implements Estado {
    @Override
    public void manejar(Contexto contexto) {
        System.out.println("Manejando el estado A...");
        System.out.println("Lógica completada. Transicionando automáticamente al Estado B.");

        // ¡LA MAGIA OCURRE AQUÍ! El propio estado le dice al contexto que

```

```
cambie.
    contexto.setEstado(new EstadoConcretoB());
}
}

// Implementación concreta del estado B
public class EstadoConcretoB implements Estado {
    @Override
    public void manejar(Contexto contexto) {
        System.out.println("Manejando el estado B...");
        System.out.println("Fin del ciclo. No hay más transiciones.");
        // Podría volver al estado A, o a un estado C, pero aquí termina.
    }
}

// Contexto que utiliza el estado
public class Contexto {
    private Estado estado;

    // El contexto arranca con un estado inicial por defecto
    public Contexto() {
        this.estado = new EstadoConcretoA();
    }

    // El setter generalmente lo usan los propios Estados, no el Main
    public void setEstado(Estado estado) {
        this.estado = estado;
    }

    public void solicitarAccion() {
        // Delega la acción al estado actual
        estado.manejar(this);
    }
}

// Uso del patrón
public class Main {
    public static void main(String[] args) {
        // 1. Creamos el contexto. Por defecto arranca en Estado A.
        Contexto contexto = new Contexto();

        System.out.println("--- Primera llamada ---");
        // 2. El cliente solo pide una acción. No tiene idea de qué estado
        // interno hay.
        contexto.solicitarAccion();
        // Salida: Manejando el estado A...
        // Salida: Lógica completada. Transicionando automáticamente al
        // Estado B.

        System.out.println("\n--- Segunda llamada ---");
        // 3. El cliente vuelve a pedir la misma acción, pero el
        // comportamiento cambió.
        contexto.solicitarAccion();
        // Salida: Manejando el estado B...
```

```
        // Salida: Fin del ciclo. No hay más transiciones.  
    }  
}
```

...

3. Metodologías de Desarrollo y Testing

TDD (Test Driven Development)

El **TDD (Desarrollo Guiado por Pruebas)** es una técnica impulsada fuertemente por la metodología *eXtreme Programming (XP)* creada por Kent Beck. Combina el desarrollo de pruebas primero (*Test First Development*) con la refactorización continua. Su objetivo es pensar en el diseño y en los requerimientos antes de escribir el código de producción, utilizando los tests como "contratos ejecutables". En TDD, **la arquitectura emerge desde el código** a medida que se resuelven los problemas, en lugar de diseñarse estáticamente de antemano.

El Ciclo de TDD (Red - Green - Refactor - Repeat):

1. **Red (Falla):** Escribir un test que falle para una nueva funcionalidad o requerimiento. El rojo indica que hay trabajo por hacer.
2. **Green (Pasa):** Escribir apenas el código funcional mínimo necesario para que ese test pase.
3. **Refactor:** Mejorar la calidad y el diseño del código (eliminar duplicación, aplicar patrones), comprobando que los tests sigan en verde.
4. **Repeat:** Repetir el ciclo iterativamente con el siguiente caso de prueba.

Granularidad de los Tests:

- **Test de Unidad (Unit Tests):** Aíslan cada unidad mínima del programa (un método o clase) para probarla. Se escriben desde la perspectiva del programador.
- **Test de Aceptación (Acceptance Tests):** Prueban funcionalidades completas desde la perspectiva del cliente.

Patrones de Pruebas: Test Doubles

Al hacer pruebas de unidad sobre un Sistema Bajo Prueba (*SUT - System Under Test*), frecuentemente este depende de otros objetos o módulos (*DOC - Depended-On Component*) que no están disponibles, son muy lentos (ej. una base de datos real) o cuyos errores queremos aislar.

Para solucionar esto, reemplazamos el componente real por una maqueta polimórfica (un "doble de riesgo" o **Test Double**) que ofrece la misma interfaz. El catálogo de Test Doubles se clasifica según su nivel de complejidad e intención:

- **Dummy Object (Objeto Muñeco):** Es un objeto que se pasa como parámetro a un método (para satisfacer la firma) pero que nunca es utilizado o invocado por el SUT.
- **Test Stub (Talón):** Es un "cascarón vacío" que provee respuestas predefinidas o enlatadas (*indirect inputs*) a las llamadas que recibe del SUT durante la prueba. Su objetivo principal es que el SUT pueda compilar y ejecutar su camino de código.
- **Test Spy (Espía):** Es un *Stub* que, además de devolver respuestas, guarda un registro interno de los mensajes, llamados y salidas indirectas que recibe del SUT. El test luego le "pregunta" al espía qué fue

lo que ocurrió para hacer aserciones.

- **Mock Object (Objeto Burlón):** Es un doble más complejo que no solo registra, sino que **verifica activamente** si los mensajes recibidos son correctos. El propio Mock tiene programadas las expectativas (qué métodos deben llamarse, en qué orden y con qué parámetros) y falla la prueba si el SUT no cumple con ese contrato.
- **Fake Object (Imitación):** Es una simulación ligera que tiene una implementación real y funcional (protocolos, tiempos de respuesta), pero simplificada para el entorno de pruebas. Un ejemplo clásico es reemplazar una base de datos real pesada por una base de datos falsa implementada con tablas Hash en memoria para que los tests corran rapidísimo.

Tip para el examen: La diferencia clave entre Spy y Mock es que el Spy solo anota lo que pasó para que el test lo verifique después, mientras que el Mock ya sabe lo que "debería" pasar y él mismo realiza la verificación y falla si no se cumple.



4. Frameworks

Conceptos Generales y Diferencias con Bibliotecas

Un framework es una **aplicación "semi-completa" y reusable**, compuesta por un conjunto de clases concretas y abstractas que proveen una arquitectura para implementar una familia de aplicaciones relacionadas. La principal diferencia entre una librería (biblioteca) y un framework radica en el **hilo de control (Inversión de Control o Hollywood Principle)**:

- **Librería:** Nuestro código "llama" a la funcionalidad de la librería (ej. conectarse a una base de datos).
- **Framework:** Provee una arquitectura de ejecución. Es el framework quien contiene el flujo central (bucle de control) e invoca a nuestro código en momentos específicos ("no nos llames, nosotros te contactaremos").

Consideraciones de Diseño: FrozenSpots y HotSpots

El diseño de un framework se divide en partes inalterables y partes flexibles:

- **FrozenSpots:** Son los aspectos del diseño que **no cambian** ("marca indeleble"). Representan el comportamiento central y afectan a todas las instanciaciones (ej. el bucle infinito que escucha conexiones en un servidor TCP).
- **HotSpots:** Son las estructuras en el código que permiten modificar o adaptar el comportamiento del framework para instanciarlo o extenderlo. Suelen implementarse a través de *Hook Methods* (métodos gancho) o mediante Patrones de Diseño.

Tipos de Instanciación: Caja Blanca y Caja Negra

Existen dos formas principales de instanciar (usar) un framework según cómo se acople nuestro código a él:

- **Frameworks de Caja Blanca (White-Box):**
 - La instanciación se logra fundamentalmente a través de la **herencia estática**.
 - El desarrollador crea subclases de las clases del framework y sobrescribe *hook methods* para "completar" el hilo de control.
 - **Desventaja:** Exige que el programador tenga un conocimiento profundo sobre el código fuente interno del framework.

- **Ejemplo visto:** `SingleThreadTCPServer`, donde cada servidor particular se crea heredando de la clase base y redefiniendo el método `handleMessage()` para inyectar su funcionalidad. El patrón de diseño subyacente más utilizado aquí es el *Template Method*.
- **Frameworks de Caja Negra (Black-Box):**
 - La instanciación se logra mediante **composición y configuración** de objetos.
 - No se requiere modificar o heredar el código fuente base; el flujo de control delega en objetos que se inyectan dinámicamente.
 - **Ejemplo visto:** `java.util.logging`, donde la configuración deseada se logra componiendo filtros, formateadores y salidas (*outputs*), o también el manejo de mensajes mediante patrones como *Strategy* o *Command* en `tcp.server.reply`.

Relación entre Frameworks y Patrones de Diseño

El diseño de frameworks está íntimamente ligado a los patrones de diseño:

- **El reuso emerge:** Existe una "Paradoja del Diseño de Frameworks". Los frameworks no se diseñan siendo reusables desde cero; el reuso *emerge* al resolver repetidamente problemas de un mismo dominio (según Ralph Johnson: "Un framework emerge de resolver al menos 3 ejemplos").
- **Los patrones son las micro-arquitecturas:** A medida que el diseño evoluciona, se adapta a los problemas comunes usando patrones de diseño.
- **Patrones como HotSpots:** Si al analizar el diseño de un framework identificamos un patrón de diseño (como *Strategy*, *Command*, *Template Method*, etc.), **esa estructura es seguramente un HotSpot** pensado para dar flexibilidad y permitir la instanciación.

...